CHAPTER 2 · APPLICATION LAYER

2.2 HTTP

The Web and HTTP

Kurose & Ross, 8th Ed.

INTRO

The Web — Objects and URLs

Section 2.2: Introduction

A Web Page = A Collection of Objects

An object is any file — HTML, JPEG, JavaScript, CSS, video clip — addressable by a single URL.

Most pages = 1 base HTML file + several referenced objects.

Example: 1 HTML + 5 JPEG images = 6 objects total.

URL = hostname + path name

http://www.someSchool.edu/dept/pic.gif

Hostname: www.someSchool.edu
Path: /dept/pic.gif

Each object has its own unique URL.

Browsers implement the HTTP client side · Web servers house objects and implement the HTTP server side

The Web operates on demand — you get what you want, when you want it

OVERVIEW

HTTP — What Is It?

Section 2.2.1: Overview of HTTP

HTTP = HyperText Transfer Protocol — the Web's application-layer protocol

Client-Server Model

Browser = HTTP client
Web server = HTTP server

Server is always on with a fixed IP, serving millions of browsers simultaneously.

Defined in:
RFC 1945 · RFC 7230 · RFC 7540

Runs Over TCP

NOT UDP — HTTP uses TCP.
Default port: 80.

Client initiates TCP connection.
Both sides communicate via socket interfaces.

HTTP gets reliable data transfer for free — no need to worry about lost packets.

Stateless Protocol

Server maintains no information about past clients.

Same object requested twice → server sends it again, as if the first request never happened.

Simplifies server design enormously → high-performance servers with thousands of simultaneous connections.

CONNECTIONS

Non-Persistent Connections (HTTP/1.0)

Section 2.2.2: Connections

Example: page with 1 HTML base file + 10 JPEG images on the same server

1. **Initiate TCP**
   Client opens a TCP connection to port 80 of the server. One socket on each side.

2. **Send HTTP Request**
   Client sends an HTTP request message for the base HTML file through its socket.

3. **Server Responds**
   Server retrieves the HTML file, wraps it in an HTTP response, sends it — then closes the TCP connection.

4. **Client Parses HTML**
   Client receives the response, TCP terminates. Parses HTML → finds 10 JPEG references.

5. **Repeat × 10**
   Steps 1–3 are repeated for each of the 10 JPEG objects → 11 TCP connections total for one page!

CONNECTIONS

RTT Analysis: Cost of Non-Persistent HTTP

Section 2.2.2: Connections

RTT (Round-Trip Time) = time for a small packet to travel from client → server → client

Time to transfer 1 object (non-persistent):

1 RTT — TCP connection initiation (SYN / SYN-ACK)

1 RTT — HTTP request + first bytes of response

Transmission time — receive the full object

Per object cost = 2 RTTs + transmission time

For 11 objects (1 HTML + 10 JPEGs):

11 × (2 RTTs + transmission time)

= 22 RTTs of TCP handshake overhead alone!

Each new TCP connection requires a fresh 3-way handshake — even for tiny objects.

This overhead motivated the design of persistent connections in HTTP/1.1.

Persistent Connections with Pipelining (HTTP/1.1)

CONNECTIONS

Section 2.2.2: Connections

Non-Persistent (HTTP/1.0)

New TCP connection for every object.

Cost: 2 RTTs per object.

For a page with 11 objects:
→ 22 RTTs of TCP handshake overhead.

Server must open and manage 11 separate TCP connections.

Persistent + Pipelining (HTTP/1.1)

Server leaves TCP connection open after response.

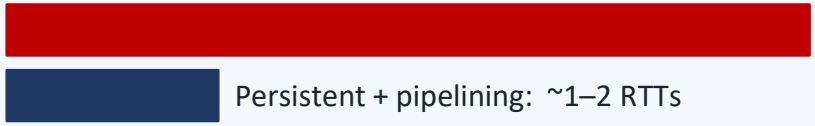
All objects on same server share one connection.

Pipelining: client sends requests back-to-back without waiting for replies.

Server sends objects back-to-back.

Connection closes after configurable idle timeout.

Performance gain with persistent + pipelining:



MESSAGES

HTTP Request Message

Section 2.2.3: HTTP Message Format

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr

(blank line – end of headers)
(entity body – empty for GET)
```

Request Line

Method + URL + HTTP version
→ 3 fields on the first line

Headers

Host:
Hostname of the server — required for proxy caches

User-agent:
Browser type/version — server can tailor its response

Accept-language:
Content negotiation — preferred language if available

Blank line

End of headers

Entity body

It contains the requested object itself

Message is plain ASCII text — human readable · Lines end with CR LF

MESSAGES

HTTP Request Methods

Section 2.2.3: HTTP Message Format

GET

Request an object identified by the URL. Entity body is empty. The vast majority of all HTTP requests use GET.

POST

Submit form data to the server. The input data is placed in the entity body (not the URL). Used for search engines, login forms, etc.

HEAD

Like GET, but the server responds with headers only — no object body. Useful for debugging and checking if an object exists.

PUT

Upload an object to the specific URL path on the server. Allows users to publish content.

DELETE

Delete the object specified in the URL from the server.

Note: HTML forms can use either GET or POST. With GET, form data is embedded directly in the URL (e.g., /search?q=networks). With POST, data goes in the entity body — not visible in the URL.

HTTP Response Message

Section 2.2.3: HTTP Message Format

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html

(blank line)
(data  data  data  ...  the HTML file)
```

Status Line
HTTP version + status code + status phrase

Headers

Date:
Date/time the response was created by the server

Server:
Server software (analogous to User-agent in requests)

Last-Modified:
When object was last modified — critical for caching

Content-Length:
Number of bytes in the entity body

Blank line
End of headers

Content-Type:
MIME type of the object (text/html, image/jpeg, ...)

Try it live: telnet gaia.cs.umass.edu 80 → GET /kurose_ross/interactive/index.php HTTP/1.1

MESSAGES

HTTP Response Status Codes

Section 2.2.3: HTTP Message Format

200 OK

Request succeeded. The requested object is included in the response body.

301 Moved Permanently

The object has moved to a new URL (given in the Location: header). Browser automatically redirects.

304 Not Modified

Response to a conditional GET. Cached copy is still valid — server sends no body. Saves bandwidth.

400 Bad Request

Generic error: the request message could not be understood or was malformed.

404 Not Found

The requested object does not exist on this server.

505 Version Not Supported

The server does not support the HTTP protocol version used in the request.

COOKIES

Cookies: Four Components

Section 2.2.4: User-Server Interaction

HTTP is stateless — but sites need to track users. Solution: Cookies (RFC 6265)

1 Set-cookie: header in HTTP response

Server creates a unique user ID and sends it to the browser in the response header.

Example: Set-cookie: 1678

2 Cookie: header in HTTP requests

On every subsequent request to the same site, the browser includes the cookie ID in the request header.

Example: Cookie: 1678

3 Cookie file on the client

The browser maintains a cookie file — one entry per site, storing the identification number assigned by that server.

4 Back-end database at the server

The server maintains a database indexed by cookie ID. It stores the user's activity, preferences, purchases, and profile.

COOKIES

Cookies in Action: The Amazon Example

Section 2.2.4: User-Server Interaction

1. **First Visit**
   Susan visits Amazon.com for the first time. Amazon has no record of her.

2. **Server Creates ID**
   Amazon creates unique ID 1678 and a database entry. Sends Set-cookie: 1678 in the HTTP response.

3. **Browser Stores Cookie**
   Susan's browser sees the Set-cookie header and writes amazon.com → 1678 to her cookie file.

4. **Subsequent Requests**
   Every later request to Amazon includes Cookie: 1678. Amazon knows which pages Susan visits, in what order, and when.

5. **One Week Later**
   Susan returns. Browser still sends Cookie: 1678. Amazon shows personalized recommendations and enables one-click checkout.

Privacy note: cookies + user-supplied info allow sites to build detailed profiles and potentially sell them to third parties — hence GDPR cookie consent banners.

CACHING

Web Caching (Proxy Server)

Section 2.2.5: Web Caching

A Web cache satisfies HTTP requests on behalf of an origin server — without contacting the origin

How a request flows through a cache:

1. Browser sends HTTP request to the cache (not the origin server).
2. Cache checks local storage. If HIT → returns the object immediately. Origin never contacted.
3. If MISS → cache opens TCP to origin server and forwards the request.
4. Origin sends the object to the cache. Cache stores a local copy and forwards it to the browser.

Why deploy caches?

① Reduced response time

Cache is close to the client (same campus LAN).
High-speed connection → object delivered fast.
Origin server's distance becomes irrelevant for cached objects.

② Reduced access link traffic

Fewer requests cross the institution's expensive Internet link.
Lower traffic intensity → less queuing delay.
Lower cost: no need to upgrade the access link bandwidth.

Cache acts as both a server (toward the browser) and a client (toward the origin server)

CACHING

Web Caching: Quantitative Analysis

Section 2.2.5: Web Caching

Setup: LAN = 100 Mbps · Access link = 15 Mbps · Avg object = 1 Mbit · Request rate = 15 req/s · Internet delay = 2 s

No Cache

Traffic intensity on access link:
= (15 req/s × 1 Mbit) / 15 Mbps = 1.0

Intensity = 1.0 → link is saturated!
Queuing delay → INFINITE

User experience: unbearable.

Upgrade Access Link to 100 Mbps

Traffic intensity:
= 15 Mbps / 100 Mbps = 0.15

Negligible queuing delay.

Response time ≈ 2 seconds
(Internet delay dominates)

Downside: expensive monthly cost.

Install Cache (hit rate = 0.4)

40% served by cache: ~10ms delay
60% still go to Internet, but:
traffic intensity = 0.6 → ~10ms

Avg delay:
= 0.4×(0.01s) + 0.6×(2.01s)
≈ 1.2 seconds

Better than Option 2, at a fraction of the cost!

Caches often run on inexpensive PCs using open-source software — much cheaper than upgrading the access link

CACHING

The Conditional GET

Section 2.2.5: Web Caching

Problem: a cached copy might be stale — the origin server may have updated the object since it was cached.

A GET is a conditional GET when it includes: If-Modified-Since: <date>

Day 1 **Cache fetches object**

Origin server responds with 200 OK + Last-Modified: Wed, 9 Sep 2015 09:23:24
Cache stores the object AND the last-modified timestamp.

Day 8 **Conditional GET sent**

Browser requests same object. Cache sends:
GET /fruit/kiwi.gif HTTP/1.1
If-modified-since: Wed, 9 Sep 2015 09:23:24

Case A **Object not changed → 304**

Server replies HTTP/1.1 304 Not Modified (empty body)
No object transferred — saves bandwidth.
Cache serves its local copy to the browser.

Case B **Object changed → 200 OK**

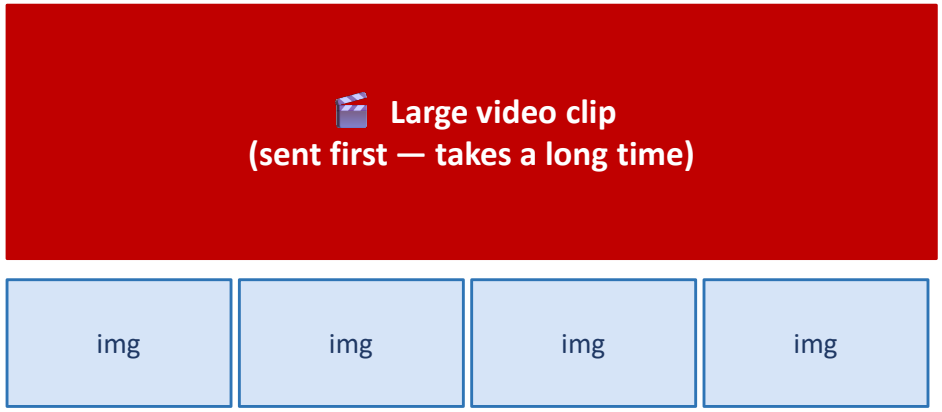
Server replies with 200 OK + new object.
Cache stores the new version and forwards it to the browser.

HTTP/1.1 Problem: Head-of-Line (HOL) Blocking

Section 2.2.6: HTTP/2

HTTP/1.1 default: one persistent TCP connection per Web page — but this introduces a critical bottleneck.

One TCP connection — objects sent in order:



Why HOL blocking is a problem

A large object at the head of the queue blocks all smaller objects behind it — even if the link is free.

Users see delays in rendering images and scripts that are waiting behind a large video or file.

HTTP/1.1 Workaround

Browsers open up to 6 parallel TCP connections.

This bypasses HOL blocking, but:
→ Wastes server socket resources.
→ Unfairly grabs extra bandwidth from TCP congestion control.

HTTP/2 was designed to fix this properly.

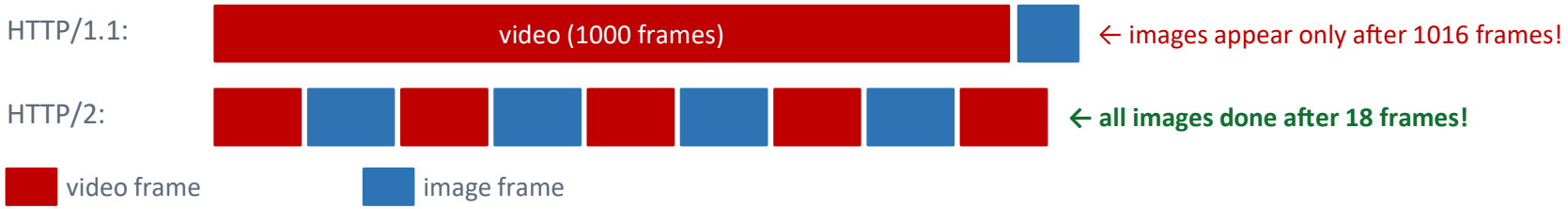
HTTP/2

HTTP/2: Framing, Prioritization and Server Push

Section 2.2.6: HTTP/2

HTTP/2 solution: break messages into frames and interleave them on ONE TCP connection

Interleaving frames — 1 video (1000 frames) + 8 images (2 frames each):



Framing

Each HTTP message is broken into small binary frames. Frames from different messages are interleaved. Reassembled at the client by the framing sub-layer.

Prioritization

Clients assign weights (1–256) to requests and declare dependencies. Server sends high-priority frames first to optimize perceived load time.

Server Push

Server can proactively push objects the client will need (found in the HTML page) — before the client even asks. Eliminates extra RTTs.

Summary

2.2 The Web and HTTP

- HTTP is the Web's application-layer protocol. It uses TCP (port 80), is stateless, and follows the client-server model.
- Web pages = collections of objects (HTML, images, JS…), each identified by a URL (hostname + path).
- Non-persistent HTTP (HTTP/1.0): 2 RTTs per object. Persistent + pipelining (HTTP/1.1): 1 connection for all objects.
- HTTP request: Request Line (method + URL + version) + header lines. Key methods: GET, POST, HEAD, PUT, DELETE.
- HTTP response: Status Line (version + code + phrase) + headers + body. Key codes: 200, 301, 304, 400, 404, 505.
- Cookies (RFC 6265): 4-component mechanism that adds a user-session layer on top of stateless HTTP.
- Web caches reduce response time AND access-link traffic. A 40% hit rate can beat doubling the access link bandwidth.
- Conditional GET (If-Modified-Since / 304 Not Modified) keeps cached copies fresh without wasting bandwidth.
- HTTP/2 (RFC 7540): frames + interleaving solve HOL blocking on a single TCP connection. Adds server push & prioritization.
- HTTP/3: runs over QUIC (UDP-based) for even lower latency connection establishment.